

LAPORAN AWAL

Sistem Kolaborasi Tim Developer: Implementasi Kanban Board dan Issue Tracker



Dosen Pengampu:

Jefry Sunupurwa Asri S.Kom., M.Kom.

Disusun Oleh:

Luqman Syahreno (20240801016)

Mata Kuliah:

Pemrograman Web (CR002)

FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

2026

BAB 1: PENDAHULUAN

1.1 Latar Belakang (Analisis Masalah & Kebutuhan Sistem)

Dalam siklus pengembangan perangkat lunak, kolaborasi tim yang efisien sangat bergantung pada kejelasan pembagian tugas dan pelacakan *bug* atau fitur. Tim developer sering kali dihadapkan pada masalah inkonsistensi data ketika menggunakan platform manajemen tugas yang terpisah dari repositori kode utama mereka, seperti GitHub. Pengembang harus memperbarui status tugas secara manual di dua tempat yang berbeda, yang tidak hanya memakan waktu tetapi juga rawan terhadap kelalaian manusia (*human error*), sehingga menyebabkan asinkronisasi antara progres aktual dan progres yang tercatat.

Berdasarkan analisis masalah tersebut, terdapat kebutuhan sistem untuk sebuah platform kolaborasi *in-house* yang terpusat, responsif, dan terintegrasi langsung dengan ekosistem kontrol versi. Kebutuhan utamanya mencakup antarmuka visual berupa Kanban Board yang intuitif untuk kemudahan operasional (*drag-and-drop*), serta sistem otomatisasi yang menjembatani komunikasi data dua arah antara basis data lokal dan GitHub. Dengan arsitektur TALL Stack, pengembangan antarmuka reaktif dapat dicapai tanpa kompleksitas pembuatan API internal untuk *client-side*, sehingga memangkas estimasi pengerjaan.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam pengembangan sistem ini adalah:

1. Bagaimana membangun sistem kolaborasi tim developer yang efisien menggunakan arsitektur TALL Stack (Laravel 12, Filament v3, Livewire 3, dan Alpine.js)?
2. Bagaimana merancang dan mengimplementasikan antarmuka Kanban Board yang interaktif dan reaktif untuk manajemen *Issue Tracker*?
3. Bagaimana menerapkan sinkronisasi dua arah (*Two-Way Sync*) antara sistem web lokal dan GitHub menggunakan GitHub REST API dan Webhook, sekaligus memitigasi risiko *infinite loop* pada pembaruan otomatis?

1.3 Tujuan

Tujuan utama dari pengerjaan proyek ini adalah:

1. Menghasilkan aplikasi web kolaborasi tim yang stabil dan terstandarisasi untuk memenuhi luaran mata kuliah Pemrograman Web.
2. Mengimplementasikan fitur manajemen tugas visual melalui Kanban Board yang didukung oleh SortableJS untuk interaktivitas *drag-and-drop*.
3. Membangun jalur integrasi pihak ketiga yang sukses menghubungkan *action* di aplikasi (dari Laravel ke GitHub via REST API) dan *action* di repositori (dari GitHub ke Laravel via Webhook) secara *real-time*.

1.4 Batasan Masalah

Agar fokus pengerjaan tetap terarah dan sesuai dengan estimasi waktu 4-5 minggu, proyek ini dibatasi pada:

1. **Teknologi:** Backend dan routing menggunakan Laravel 12 (PHP 8.2+). Admin panel dan manajemen master data menggunakan Filament v3.3. Frontend di-handle oleh Blade, Livewire 3, Alpine.js, dan Tailwind CSS.
2. **Pengguna:** Sistem otorisasi mencakup manajemen peran dasar, dibatasi pada *Project Manager* dan *Developer*.
3. **Fitur Utama:** Kanban Board difokuskan pada empat kolom status (*To Do*, *In Progress*, *Review*, *Done*). *Issue tracker* mencakup pelabelan tugas dasar (*bug* atau *feature*).
4. **Integrasi:** Sinkronisasi terbatas pada *Issue* di GitHub. Autentikasi menggunakan *Personal Access Token* (PAT), dan *endpoint* Webhook akan diamankan dengan validasi *sender login* pada *payload* JSON.

1.5 Manfaat

1. **Bagi Tim Developer:** Meningkatkan transparansi alur kerja dan mengurangi beban kerja administratif karena pembaruan status tugas (*issue*) berjalan secara otomatis.
2. **Bagi Pengembang Sistem:** Memberikan pemahaman praktis dan mendalam mengenai implementasi *state management* reaktif dengan Livewire, serta pemrosesan *payload* dari API pihak ketiga secara aman di dalam ekosistem Laravel.

1.6 Kerangka Berpikir Penelitian

Pengembangan dimulai dari identifikasi masalah (inkonsistensi pelacakan tugas), dilanjutkan dengan analisis kebutuhan (Sistem Kanban terintegrasi GitHub), pemilihan teknologi pendorong efisiensi (TALL Stack & Filament), perancangan basis data dan *endpoint* API, implementasi fitur inti (CRUD & Kanban *Drag-and-drop*), pembuatan jalur sinkronisasi (REST API & Webhook), dan diakhiri dengan pengujian fungsionalitas serta *handling error* (Mitigasi *infinite loop*).

1.7 Metodologi Ringkas

Pendekatan penyelesaian masalah menggunakan metode *Agile* dengan iterasi pendek. Tahapan dimulai dari perancangan arsitektur data (*Entity Relationship Diagram*), dilanjutkan dengan setup *boilerplate* Laravel 12 dan lingkungan Docker. Fase *development* akan dibagi menjadi penyelesaian modul *master data* via Filament, pembuatan antarmuka Kanban dengan Livewire/Alpine.js, dan diakhiri dengan fase integrasi dan pengujian komunikasi *Two-Way Sync* GitHub menggunakan *tunneling* lokal (seperti Ngrok/Expose).

BAB 2: STUDI TEORITIS

2.1 Konsep Kanban dan Manajemen Tugas

Kanban adalah sebuah kerangka kerja visual yang digunakan untuk mengelola dan memantau pekerjaan yang bergerak melalui suatu proses. Dalam konteks pengembangan perangkat lunak, papan Kanban biasanya dibagi menjadi beberapa kolom status seperti *To Do*, *In Progress*, *Review*, dan *Done*. Penggunaan Kanban memungkinkan tim untuk memvisualisasikan beban kerja, mengidentifikasi hambatan (*bottleneck*), dan memastikan transparansi status setiap *issue* atau tugas secara *real-time*.

2.2 Arsitektur TALL Stack

TALL Stack adalah pendekatan modern dalam pengembangan aplikasi web yang menggabungkan empat teknologi utama untuk menciptakan aplikasi reaktif *full-stack* dengan efisiensi tinggi.

- **Tailwind CSS (v3.4+)**: Sebuah *framework* CSS berbasis utilitas (*utility-first*) yang memungkinkan penataan gaya antarmuka secara langsung pada *markup* HTML atau Blade *templates* tanpa perlu menulis baris CSS kustom.
- **Alpine.js**: *Framework* JavaScript yang ringan untuk menambahkan perilaku interaktif pada sisi klien (seperti *dropdown*, modal, atau inisialisasi *library* eksternal) dengan sintaks yang deklaratif langsung di dalam HTML.
- **Laravel 12 (PHP 8.2+)**: *Framework* backend PHP yang menangani *routing* utama, logika bisnis, keamanan, dan interaksi dengan basis data. Laravel bertindak sebagai fondasi utama aplikasi.
- **Livewire 3**: *Framework* untuk Laravel yang memungkinkan pembuatan antarmuka dinamis dan reaktif. Livewire menangani sinkronisasi *state* antara komponen *frontend* dan *backend* menggunakan metode AJAX secara *silent*, mengeliminasi kebutuhan untuk membangun API RESTful secara manual hanya untuk komunikasi antarmuka klien.

2.3 Filament PHP (v3.3)

Filament adalah sekumpulan *tools* dan komponen *full-stack* untuk Laravel yang dirancang untuk mempercepat pembuatan antarmuka administratif (Admin Panel). Dibangun di atas fondasi TALL Stack, Filament menyediakan sistem *boilerplate* yang kuat untuk manajemen *resource* (CRUD), otorisasi peran (*Role Management* via *plugin* seperti Shield), dan antarmuka tabel serta formulir yang terstandarisasi. Dalam proyek ini, Filament menopang sistem *master data* dan pengaturan *workspace* proyek.

2.4 Integrasi API dan Webhook (Sinkronisasi Dua Arah)

Untuk mencapai fitur *Two-Way Sync* dengan ekosistem GitHub, sistem mengimplementasikan dua jalur komunikasi arsitektural:

- **REST API (Representational State Transfer):** Arsitektur perangkat lunak yang menggunakan protokol HTTP untuk pertukaran data. Sistem ini memanfaatkan GitHub REST API dengan autentikasi *Personal Access Token* (PAT) untuk mengirimkan perintah POST/PATCH dari aplikasi lokal (Laravel) guna membuat atau memperbarui status *issue* secara langsung di repositori GitHub.
- **Webhook:** Mekanisme pengiriman pesan otomatis (berupa *payload* JSON) antar sistem yang dipicu oleh suatu peristiwa (*event-driven*). GitHub Webhook dikonfigurasi untuk memonitor perubahan pada repositori dan mengirimkan data pembaruan ke *Route Endpoint* publik Laravel. Logika di dalam *controller* akan memvalidasi *sender* untuk mencegah *infinite loop* sebelum memperbarui basis data lokal.

2.5 SortableJS

SortableJS adalah pustaka JavaScript independen yang ringan dan berfungsi untuk membuat daftar elemen HTML dapat diurutkan melalui mekanisme *drag-and-drop*. Dalam arsitektur sistem ini, SortableJS diinisialisasi melalui Alpine.js pada komponen Kanban Board. Saat kartu tugas dipindahkan antar kolom, perpindahan tersebut memicu fungsi Livewire untuk memperbarui status di basis data (dan selanjutnya ke GitHub) secara *real-time*.

2.6 Arsitektur Lingkungan Pengembangan (Docker)

Untuk memastikan konsistensi lingkungan antara tahap pengembangan dan *deployment*, proyek ini mengadopsi arsitektur berbasis kontainer menggunakan Docker. Melalui konfigurasi `docker-compose.yml`, arsitektur dipisahkan menjadi beberapa layanan spesifik (seperti *Nginx* untuk *web server*, *PHP container* untuk eksekusi Laravel/Composer, dan *Database container* untuk manajemen data), yang memastikan dependensi aplikasi berjalan pada lingkungan yang terisolasi dan terstandarisasi.

BAB 3: METODOLOGI PENELITIAN

3.1 Pendekatan Pengembangan Sistem

Penelitian dan pengembangan sistem ini menggunakan pendekatan *Agile Kebijakan* dengan model pengembangan iteratif (*Iterative Development*). Mengingat estimasi waktu pengerjaan yang terbatas (4 hingga 5 minggu), metodologi ini dipilih karena sifatnya yang adaptif terhadap perubahan dan berfokus pada penyampaian fungsionalitas produk secara bertahap namun konsisten.

Proses pengembangan dibagi menjadi siklus-siklus kecil (*sprints*) mingguan, di mana setiap akhir siklus akan menghasilkan komponen fitur yang dapat diuji secara mandiri. Pendekatan ini memastikan bahwa arsitektur TALL Stack dan fitur integrasi pihak ketiga dapat dievaluasi secara berkala tanpa harus menunggu seluruh sistem selesai dibangun.

3.2 Tahapan Pelaksanaan Proyek

Untuk memastikan penyelesaian sistem berjalan secara terstruktur dan sesuai dengan konteks penelitian, pelaksanaan proyek dibagi ke dalam lima tahapan utama:

3.2.1 Tahap Analisis Kebutuhan (Requirements Analysis)

Pada tahap awal, dilakukan identifikasi masalah terkait manajemen tugas pada tim pengembang, khususnya asinkronisasi antara repositori kode dan papan kerja visual. Dari analisis tersebut, dirumuskan spesifikasi kebutuhan fungsional sistem, seperti kebutuhan otorisasi pengguna (peran *Project Manager* dan *Developer*), kebutuhan antarmuka Kanban Board yang reaktif, serta kebutuhan spesifikasi Route API untuk menangani *payload* dari pihak ketiga.

3.2.2 Tahap Perancangan Sistem (System Design)

Tahap ini berfokus pada pemodelan arsitektur data dan alur logika sistem sebelum masuk ke proses pengodean. Aktivitas utamanya meliputi perancangan struktur tabel basis data untuk mengakomodasi relasi proyek, tugas, status, dan pemetaan ID unik GitHub, serta perancangan logika kontroler untuk menjembatani komunikasi data dua arah.

3.2.3 Tahap Implementasi Konstruksi (Development & Coding)

Fase konstruksi perangkat lunak dieksekusi dengan memanfaatkan efisiensi dari arsitektur TALL Stack murni dan ekosistem Laravel. Langkah-langkah pada tahap ini meliputi:

1. **Konfigurasi Lingkungan Kontainer:** Menyiapkan lingkungan pengembangan yang terisolasi menggunakan Docker (*nginx*, *php*, dan *db*) untuk memastikan konsistensi dependensi.

2. **Inisialisasi Antarmuka Administrator:** Menggunakan *boilerplate* Filament v3 untuk membangun modul *master data* (User, Role, dan Project) secara cepat.
3. **Penyusunan User Workspace:** Membangun antarmuka Kanban Board visual pada Blade Templates dengan memanfaatkan Livewire 3 untuk manajemen *state* reaktif dan Alpine.js untuk inisialisasi pustaka SortableJS pada fitur *drag-and-drop*.

3.2.4 Tahap Integrasi Pihak Ketiga (Third-Party Integration)

Tahap ini merupakan penerapan nilai tambah dari sistem, yaitu sinkronisasi dua arah dengan GitHub. Proses integrasi dibagi menjadi dua jalur:

1. **Outbound Sync (Laravel ke GitHub):** Implementasi HTTP Client Laravel untuk menembak GitHub REST API menggunakan *Personal Access Token* (PAT) setiap kali terjadi perubahan status tugas lokal.
2. **Inbound Sync (GitHub ke Laravel):** Penyusunan Route API Endpoint publik yang dibantu oleh alat *tunneling* (seperti Ngrok atau Expose) untuk menangkap payload JSON dari GitHub Webhook. Pada tahap ini juga diimplementasikan logika validasi *sender login* untuk menghentikan proses jika pembaruan dipicu oleh sistem itu sendiri.

3.2.5 Tahap Pengujian dan Validasi (Testing)

Pengujian dilakukan secara berkelanjutan menggunakan kerangka kerja pengujian Pest PHP untuk memastikan validitas kode backend dan fungsionalitas komponen Livewire. Selain pengujian unit (*unit testing*), dilakukan juga pengujian integrasi secara manual terhadap respons Webhook guna memastikan bahwa mekanisme mitigasi *infinite loop* berfungsi dengan sempurna ketika sinkronisasi otomatis berjalan.

3.3 Teknik Pengumpulan Data dan Informasi

Informasi yang mendasari pengembangan sistem ini dikumpulkan melalui dua teknik utama:

1. **Studi Literatur dan Dokumentasi Resmi:** Mempelajari dokumentasi resmi Laravel 12, Filament PHP, Livewire v3, serta spesifikasi teknis GitHub REST API dan Webhook Event Developer Guides.
2. **Analisis Alur Kerja Terbuka (*Open-Source Workflow Analysis*):** Mengamati pola manajemen *issue* dan pelabelan tugas yang standar pada platform repositori bersama untuk diterapkan ke dalam logika kolom Kanban Board (*To Do*, *In Progress*, *Review*, *Done*).

3.4 Lingkungan Pengendalian Kualitas Kode

Untuk menjaga konsistensi, kerapian, dan kualitas kode selama masa pengembangan 4-5 minggu, proyek ini menerapkan standarisasi otomatis menggunakan Laravel Pint. Penggunaan alat pemformatan kode (*code formatter*) ini memastikan seluruh sintaks PHP yang ditulis

mematuhi panduan gaya (*style guide*) yang baku, mengurangi potensi kesalahan sintaksis, dan mempermudah proses penelaahan kode (*code review*).

BAB 4: HASIL DAN PEMBAHASAN (RENCANA PERANCANGAN)

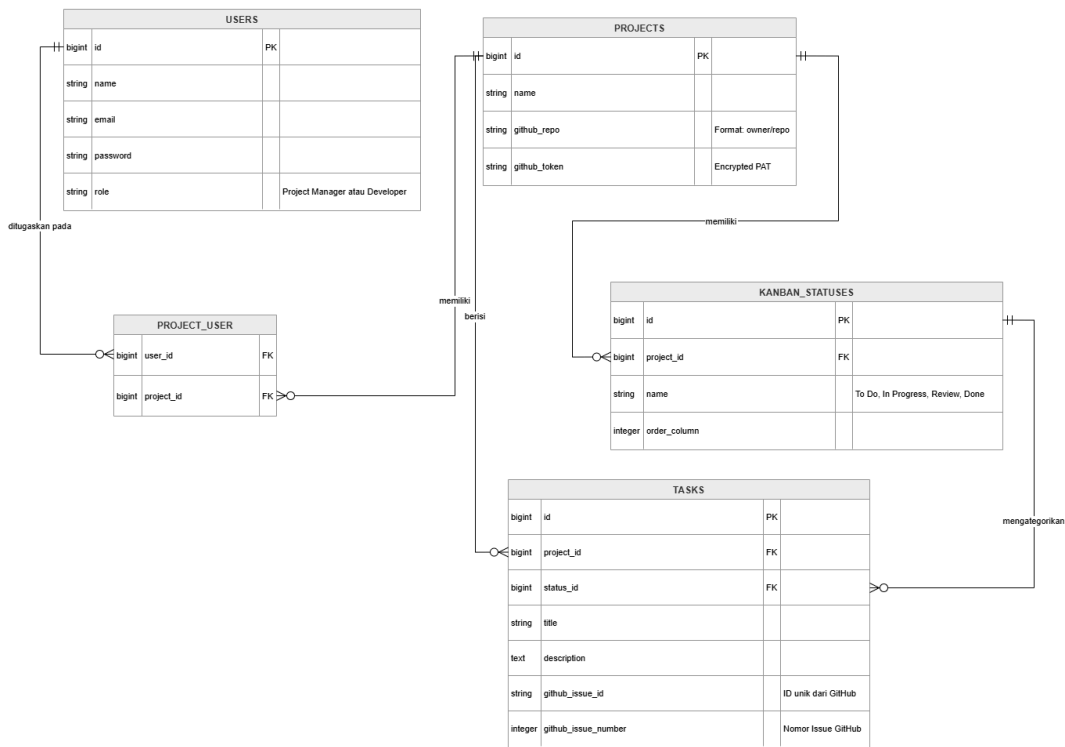
4.1 Rencana Hasil Pengembangan Sistem

Hasil akhir yang direncanakan dari pengerjaan Tugas Akhir ini adalah sebuah aplikasi web "Developer Team Collaboration System" yang stabil dan siap digunakan untuk manajemen proyek. Luaran sistem direncanakan memiliki spesifikasi sebagai berikut:

- 1. **Antarmuka Visual:** Papan Kanban interaktif dengan responsivitas tinggi menggunakan Livewire, yang memungkinkan pemindahan tugas secara *drag-and-drop*.
- 2. **Dashboard Administrator:** Panel manajemen pusat yang efisien untuk mengelola pengguna, peran, dan sinkronisasi repositori GitHub melalui Filament v3.
- 3. **Otomasi Sinkronisasi:** Jalur komunikasi data yang aman dan otomatis antara pangkalan data lokal dan GitHub Issue Tracker.

4.2 Perancangan Basis Data (Entity Relationship Diagram)

Untuk mendukung rencana hasil di atas, dirancang sebuah skema basis data yang mampu menyimpan data kolaborasi sekaligus memetakan identitas unik dari pihak ketiga (GitHub). Berikut adalah rancangan *Entity Relationship Diagram* (ERD) sistem:

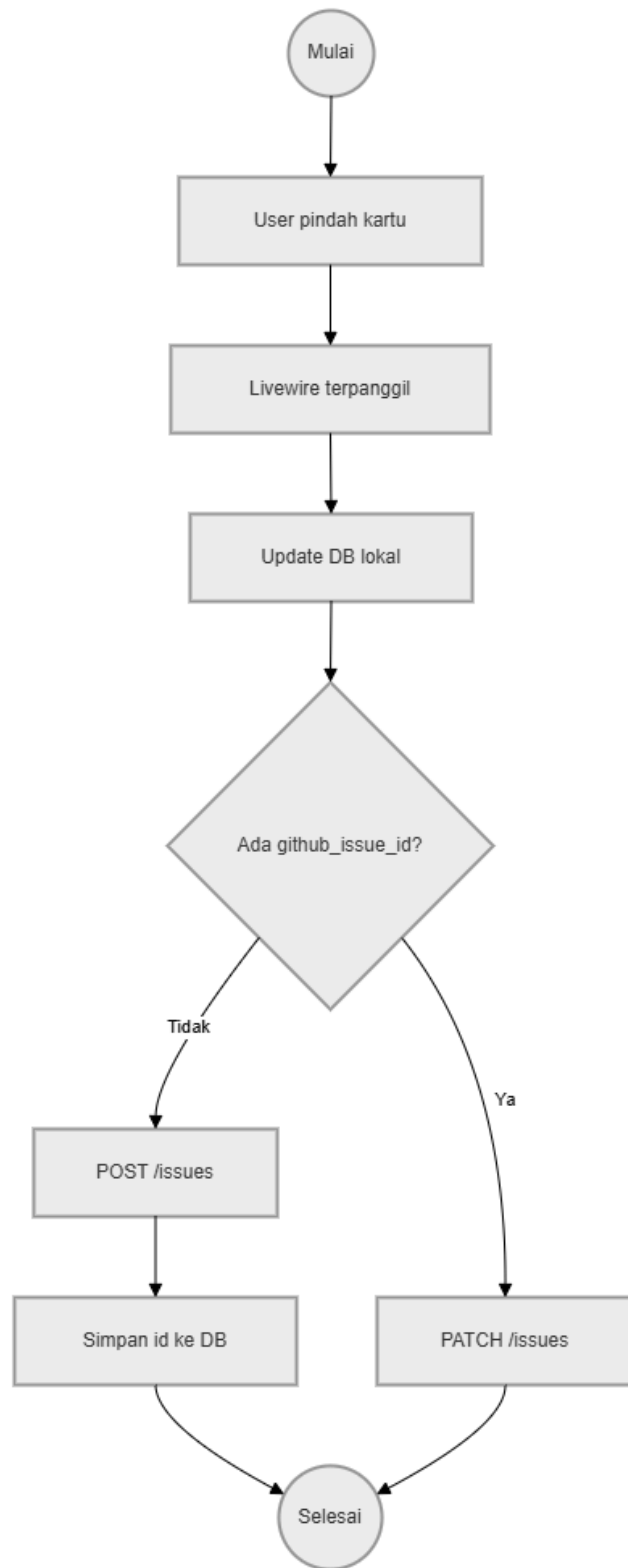


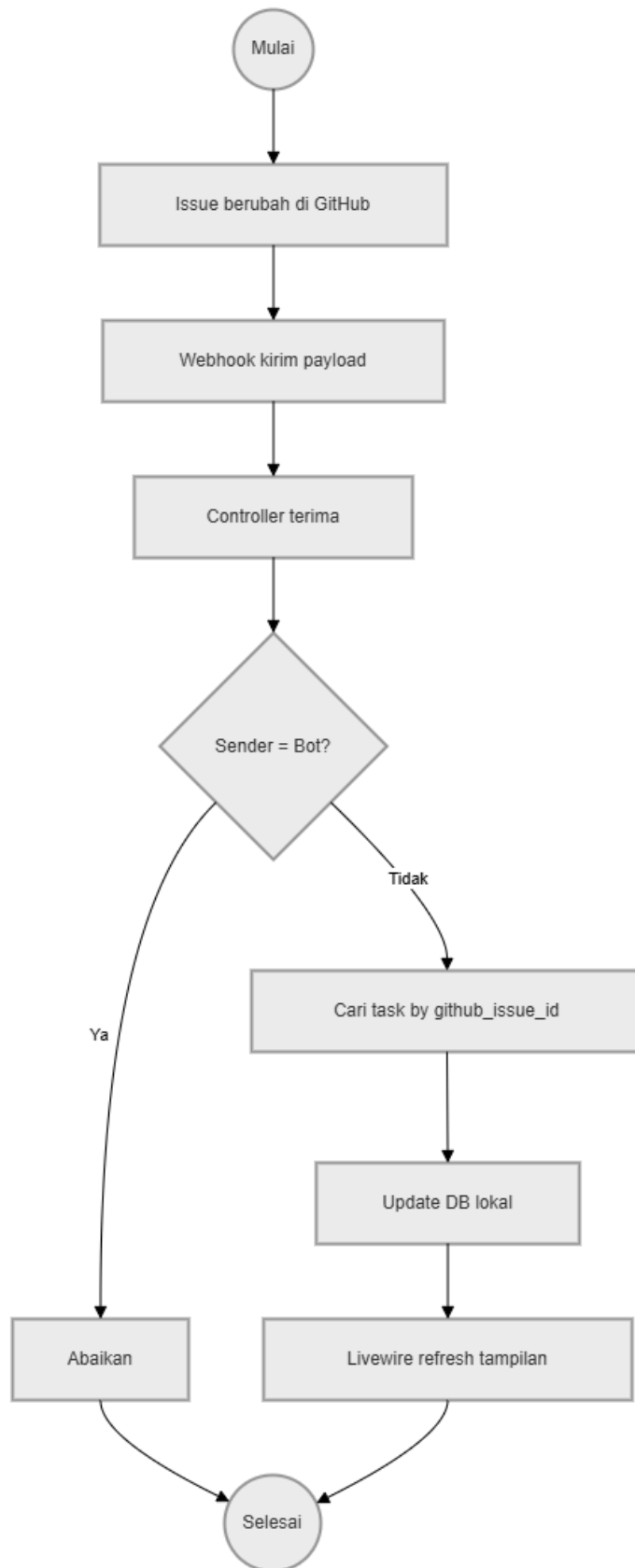
Penjelasan Rancangan ERD:

1. **Tabel `projects`**: Berfungsi menyimpan informasi krusial untuk integrasi, yaitu alamat repositori dan token akses (PAT) yang dienkripsi untuk keamanan.
2. **Tabel `tasks`**: Bertindak sebagai *Issue Tracker*. Kolom `github_issue_id` dan `github_issue_number` dirancang sebagai kunci pemetaan (mapping key). Tanpa kedua kolom ini, sinkronisasi dua arah tidak mungkin dilakukan karena sistem tidak akan tahu tugas mana yang harus diperbarui saat menerima data dari GitHub.
3. **Tabel `kanban_statuses`**: Memungkinkan setiap proyek memiliki penamaan kolom Kanban yang fleksibel namun tetap terstruktur untuk keperluan pelacakan progres.

4.3 Perancangan Alur Sistem (Flowchart Two-Way Sync)

Logika utama yang akan dikembangkan adalah sinkronisasi dua arah. Flowchart di bawah ini menjelaskan rencana alur kerja bagaimana data berpindah dari aplikasi lokal ke GitHub dan sebaliknya, termasuk mekanisme pencegahan kesalahan sistem.





Penjelasan Rencana Alur Kerja:

1. **Mekanisme Outbound:** Direncanakan agar setiap aksi *drag-and-drop* pada UI Kanban langsung memicu perintah HTTP ke GitHub. Jika tugas tersebut baru dibuat di aplikasi web, sistem akan meminta GitHub membuat nomor *issue* baru, lalu menyimpannya kembali ke pangkalan data lokal.
2. **Mekanisme Inbound (Webhook):** Sistem akan menyediakan sebuah pintu masuk (Route API) untuk menerima kabar dari GitHub.
3. **Logika Mitigasi Infinite Loop:** Ini adalah bagian krusial dari hasil yang ingin dicapai. Sistem dirancang untuk memeriksa siapa pengirim data tersebut. Jika pengirimnya adalah akun bot yang digunakan aplikasi, maka data akan diabaikan. Hal ini bertujuan agar perubahan yang dilakukan aplikasi tidak memicu Webhook yang kemudian kembali lagi ke aplikasi secara terus-menerus (*looping*).